```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include <string.h>
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <iomanip>
using namespace std;

#define rdint(i) (rand() % (int)(i))
#define rdft() (((double)rdint(7384)) / 7383.0)
#define rnd(a,b) (rdint((int)(b)-(int)(a)+1) + (int)(a))

#define POPSIZE 40                    //Population size
#define DIMENSION 100                 //Number of items
#define NUM_KNAPSACKS 10              //Number of knapsacks

#define MAXITERA 1001                 //Maximum number of iterations
#define Pm 0.3                        //Initial placement probability
#define Cr 0.98                       //Crossover probability
#define RTIME 20                      //Number of repeated runs
#define STIME 11                      //Number of times to statistics intermediate results

/***********************************************
*          Data structure definition           *
***********************************************/
struct indi {
    int u[DIMENSION];                 // One-dimensional vector
    int fitx;                         // Individual fitness
} individual[POPSIZE + 1], BEST;

int Cvalue[DIMENSION];                // Item value
int Wvalue[DIMENSION];                // Item weight
int index[DIMENSION];                 // Index of items sorted by value/weight ratio
int KNAPCAPACITY[NUM_KNAPSACKS];// Knapsack capacity
double rate[DIMENSION];               // Value/weight ratio
int sorted_knapsacks[NUM_KNAPSACKS]; // Index of knapsacks sorted by remaining capacity
fstream ofile;


int HUNDBEST = 0, HUNDWORST;
```

```
int BRESULT[DIMENSION];
int HundResult[RTIME][STIME];
double MeanResult[STIME];
double HUNDMEAN, HUNDSTD;

/*************************************************
*        Function prototype declaration                    *
*************************************************/
void calculateindex(void);
void initialize(void);
void evolution(void);
void greedyoptimization(int num);
void outputresult(void);
void readInput(const char* filename);

/*************************************************
*                    Main Function                                *
*************************************************/
int main()
{
    int i, generation;
    int RT = 0, rlow;
    time_t first, second;
    double usetime;

    readInput("F:\\aaaaa 学术\\待办\\多背包方向\\实例\\FK_1\\aw1.txt");
    ofile.open("F:\\aaaaa 学术\\待办\\多背包方向\\数据\\w1.txt", ios::app);
    if (!ofile)
    {
        cerr << "Error: Unable to open output file." << endl;
        exit(1);
    }

    srand((unsigned)time(NULL));
    first = clock();
    while (RT < RTIME)
    {
        calculateindex();
        initialize();
        generation = 0;
        rlow = 0;
        while (generation < MAXITERA)
        {
            evolution();
```

```
                if ((generation % (MAXITERA / (STIME - 1)) == 0) || (generation == MAXITERA
- 1))
                {
                    HundResult[RT][rlow] = BEST.fitx;
                    rlow++;
                }
                generation++;
            }
            if (BEST.fitx > HUNDBEST)
            {
                HUNDBEST = BEST.fitx;
                memcpy(BRESULT, BEST.u, DIMENSION * sizeof(int));
            }
            printf("%d, %d ", RT + 1, HUNDBEST);
            RT++;
        }
        printf("\n");
        second = clock();
        usetime = (double)(second - first) / CLOCKS_PER_SEC;
        outputresult();
        ofile << "Time=" << usetime / RTIME << endl;
        ofile << endl;
        for (i = 0; i < DIMENSION; i++)
            ofile << BRESULT[i] << ",";
        ofile << endl << endl;
        ofile.close();
        return 0;
}

/***********************************************
*       Function for reading data from file             *
***********************************************/
void readInput(const char* filename)
{
        ifstream infile(filename);
        if (!infile)
        {
            cerr << "Error: Unable to open file " << filename << endl;
            exit(1);
        }
        string dummyLine;
        getline(infile, dummyLine);
        getline(infile, dummyLine);
        for (int i = 0; i < NUM_KNAPSACKS; i++)
```

```
        infile >> KNAPCAPACITY[i];
    for (int i = 0; i < DIMENSION; i++)
        infile >> Wvalue[i] >> Cvalue[i];
    infile.close();
}

/*********************************************
*                    Output results                    *
*********************************************/
void outputresult(void)
{
    int i, j;
    int lbest;

    lbest = HundResult[0][STIME - 1];
    HUNDWORST = HundResult[0][STIME - 1];
    HUNDMEAN = (double)HundResult[0][STIME - 1];
    for (i = 1; i < RTIME; i++)
    {
        if (HundResult[i][STIME - 1] > lbest)
            lbest = HundResult[i][STIME - 1];
        if (HundResult[i][STIME - 1] < HUNDWORST)
            HUNDWORST = HundResult[i][STIME - 1];
        HUNDMEAN += (double)HundResult[i][STIME - 1];
    }
    HUNDMEAN /= RTIME;
    if (lbest != HUNDBEST)
    {
        printf("Error...! %d != %d\n", HUNDBEST, lbest);
        exit(-1);
    }
    for (i = 0; i < RTIME; i++)
    {
        for (j = 0; j < STIME; j++)
            ofile << HundResult[i][j] << ",";
        ofile << endl;
    }
    ofile << endl;
    ofile << "BEST=" << HUNDBEST << "      WORST=" << HUNDWORST << "
MEAN=" << fixed << setprecision(2) << HUNDMEAN;
    for (j = 0; j < STIME; j++)
    {
        MeanResult[j] = 0.0;
        for (i = 0; i < RTIME; i++)
```

```
                MeanResult[j] += (double)HundResult[i][j];
            MeanResult[j] /= RTIME;
        }
        if (HUNDMEAN != MeanResult[STIME - 1])
        {
            printf("Error2...");
            exit(-1);
        }
        HUNDSTD = 0.0;
        for (i = 0; i < RTIME; i++)
            HUNDSTD += (HUNDMEAN - HundResult[i][STIME - 1]) * (HUNDMEAN -
HundResult[i][STIME - 1]);
        HUNDSTD = sqrt(HUNDSTD / RTIME);
        ofile << "       STD=" << HUNDSTD << endl;
        for (j = 0; j < STIME; j++)
            ofile << MeanResult[j] << ",";
        ofile << endl;
}

/*******************************************
*        Sort items by value/weight ratio          *
*******************************************/
void calculateindex(void)
{
        int i, j, temp1;
        for (i = 0; i < DIMENSION; i++)
        {
            rate[i] = (double)Cvalue[i] / Wvalue[i];
            index[i] = i;
        }
        for (i = 0; i < DIMENSION - 1; i++)
        {
            for (j = i + 1; j < DIMENSION; j++)
            {
                if (rate[index[i]] < rate[index[j]])
                {
                    temp1 = index[i];
                    index[i] = index[j];
                    index[j] = temp1;
                }
            }
        }
}
```

```
/***********************************************
*                Initialization function                    *
***********************************************/
void initialize()
{
    int i, j, flg;
    for (i = 0; i < POPSIZE; i++)
    {
        for (j = 0; j < DIMENSION; j++)
        {
            if (rdft() < Pm)
            {
                int choice = rdint(NUM_KNAPSACKS + 1);
                individual[i].u[j] = choice;
            }
            else
            {
                individual[i].u[j] = 0;
            }
        }
        greedyoptimization(i);
    }
    flg = 0;
    for (i = 1; i < POPSIZE; i++)
        if (individual[i].fitx > individual[flg].fitx)
            flg = i;
    BEST = individual[flg];
}

/*******************************************
*                    TS-ROA                    *
*******************************************/
void greedyoptimization(int num) {
    int i, j, k;
    int fweight[NUM_KNAPSACKS] = {0}; // Used weight of each knapsack
    int remaining_capacity[NUM_KNAPSACKS]; // Remaining capacity
    bool inB[NUM_KNAPSACKS] = {false};
    int fvalue = 0;

    // 1. Repair stage
    for (k = 0; k < NUM_KNAPSACKS; k++)
        remaining_capacity[k] = KNAPCAPACITY[k];
    for (j = 0; j < DIMENSION; j++) {
        if (individual[num].u[j] >= 1 && individual[num].u[j] <= NUM_KNAPSACKS) {
```

```
                k = individual[num].u[j] - 1;
                fweight[k] += Wvalue[j];
                remaining_capacity[k] -= Wvalue[j];
            }
        }
        for (i = DIMENSION - 1; i >= 0; i--) {
            int item_idx = index[i];
            int knap_idx = individual[num].u[item_idx] - 1;
            if (individual[num].u[item_idx] != 0 && fweight[knap_idx] > KNAPCAPACITY[knap_idx]) {
                fweight[knap_idx] -= Wvalue[item_idx];
                remaining_capacity[knap_idx] += Wvalue[item_idx];
                individual[num].u[item_idx] = 0;
            }
        }

        // 2. Optimization stage
        int random_knap = rand() % NUM_KNAPSACKS + 1;
        for (j = 0; j < DIMENSION; j++) {
            if (individual[num].u[j] == random_knap) {
                k = random_knap - 1;
                fweight[k] -= Wvalue[j];
                remaining_capacity[k] += Wvalue[j];
                individual[num].u[j] = 0;
            }
        }

        for (k = 0; k < NUM_KNAPSACKS; k++)
            sorted_knapsacks[k] = k;
        for (i = 0; i < NUM_KNAPSACKS - 1; i++) {
            int max_idx = i;
            for (j = i + 1; j < NUM_KNAPSACKS; j++) {
                if (remaining_capacity[sorted_knapsacks[j]] < remaining_capacity[sorted_knapsacks[max_idx]])
                    max_idx = j;
            }
            swap(sorted_knapsacks[i], sorted_knapsacks[max_idx]);
        }

        for (int s = 0; s < NUM_KNAPSACKS; s++) {
            k = sorted_knapsacks[s];
            inB[k] = true;

            for (i = 0; i < DIMENSION; i++) {
```

```
                int item_idx = index[i];
                int current_knap = individual[num].u[item_idx];

                if ((current_knap == 0 || (current_knap >= 1 && !inB[current_knap - 1])) &&
                    (fweight[k] + Wvalue[item_idx] <= KNAPCAPACITY[k])) {

                    if (current_knap != 0) {
                        int old_k = current_knap - 1;
                        fweight[old_k] -= Wvalue[item_idx];
                        remaining_capacity[old_k] += Wvalue[item_idx];
                    }
                    individual[num].u[item_idx] = k + 1;
                    fweight[k] += Wvalue[item_idx];
                    remaining_capacity[k] -= Wvalue[item_idx];
                }
            }
        }

        // 3.  Exchange stage
        const int thre = 20;
        const int a1 = 1, b1 = 0;
        int th = a1 * NUM_KNAPSACKS + b1 * DIMENSION;
        int re = 0;
        for (i = DIMENSION - 1; i >= 0; i--) {
            int item_i = index[i];
            int knap_i = individual[num].u[item_i];
            if (knap_i == 0) continue;

            if (re <= thre && re <= th) {
                int max_pf = 0;
                int best_j = -1;
                for (j = 0; j < DIMENSION; j++) {
                    int item_j = index[j];
                    if (individual[num].u[item_j] != 0 || item_j == item_i) continue;
                    int k_i = knap_i - 1;
                    if (fweight[k_i] - Wvalue[item_i] + Wvalue[item_j] <= KNAPCAPACITY[k_i])
{
                        int profit_diff = Cvalue[item_j] - Cvalue[item_i];
                        if (profit_diff > max_pf) {
                            max_pf = profit_diff;
                            best_j = item_j;
                        }
                    }
                }
```

```
                if (max_pf > 0 && best_j != -1) {
                    int k_i = knap_i - 1;
                    fweight[k_i] += (Wvalue[best_j] - Wvalue[item_i]);
                    individual[num].u[best_j] = knap_i;
                    individual[num].u[item_i] = 0;
                    re++;
                }
            }
        }

        fvalue = 0;
        for (j = 0; j < DIMENSION; j++) {
            if (individual[num].u[j] != 0)
                fvalue += Cvalue[j];
        }
        individual[num].fitx = fvalue;
}

/***********************************************
*                          MHDE                                *
***********************************************/
void evolution(void) {
        int i, j;
        int p1, p2, p3;
        for (i = 0; i < POPSIZE; i++) {
            p1 = rdint(POPSIZE);
            while (p1 == i) p1 = rdint(POPSIZE);
            p2 = rdint(POPSIZE);
            while (p2 == i || p2 == p1) p2 = rdint(POPSIZE);
            p3 = rdint(POPSIZE);
            while (p3 == i || p3 == p1 || p3 == p2) p3 = rdint(POPSIZE);

            for (j = 0; j < DIMENSION; j++) {
                if (rdft() < Cr) {
                    int temp = (individual[p2].u[j] != 0 && individual[p2].u[j] ==
individual[p3].u[j]) ? individual[p2].u[j] : 0;
                    int hadamard = (individual[p1].u[j] != 0 && individual[p1].u[j] == temp) ?
individual[p1].u[j] : 0;
                    individual[POPSIZE].u[j] = hadamard;
                } else {
                    individual[POPSIZE].u[j] = 0;
                }
            }
            greedyoptimization(POPSIZE);
```

```
            if (individual[POPSIZE].fitx > individual[i].fitx) {
                individual[i] = individual[POPSIZE];
                if (individual[i].fitx > BEST.fitx)
                    BEST = individual[i];
            }
        }
    }
```